HW9

Group 12 - Shir, Omer & Ido

16/06/26

Group:

Group members (Name, ID, email)

- Ido Ravid, 207198177, ido.ravid@mail.huji.ac.il
- Omer Sutovsky, 318735107 ,omer.sutovsky@mail.huji.ac.il
- Shir Tsarfaty, 207718008, shir.tsarfaty@mail.huji.ac.il

Paths and Data

```
if (!require('data.table')) { install.packages('data.table'); library('data.table') }
if (!require('splines'))    { install.packages('splines');    library('splines') }
if (!require('MASS'))       { install.packages('MASS');       library('MASS') }

source("../utils.R")
source("../lab5/lab5_utils.R")

healthy_file <- "../lab2/data/TCGA-13-0723-10B_lib2_all_chr1.forward"
tumor_file   <- "../lab7/data/TCGA-13-0723-01A_lib2_all_chr1.forward"
chr1_file    <- "../lab1/data/chr1_str.rda"
```

```
healthy_reads <- fread(healthy_file, col.names = c("Chr", "Loc", "Length"))
tumor_reads   <- fread(tumor_file,   col.names = c("Chr", "Loc", "Length"))
load(chr1_file)
```

```
BIN <- 2500

add_pos <- function(df, beg) {
  df$pos <- beg + (seq_len(nrow(df)) - 1) * BIN + BIN / 2
  df
}

# Two arms around centromere (120-130 Mb excluded), both samples
df_h <- rbind(
  add_pos(bin_data(healthy_reads, chr1, 5000001L,   119990000L, BIN), 5000001L),
  add_pos(bin_data(healthy_reads, chr1, 130000001L, 242000000L, BIN), 130000001L)
)
df_t <- rbind(
  add_pos(bin_data(tumor_reads, chr1, 5000001L,   119990000L, BIN), 5000001L),
  add_pos(bin_data(tumor_reads, chr1, 130000001L, 242000000L, BIN), 130000001L)
)
```

```r
# Per-sample GC spline + plug-in estimator (lab 8 method b)
eps <- 1
gc_correct <- function(df) {
  nz   <- df$reads > 0
  fit  <- fit_gc_spline(df[nz, ])
  fhat <- pmax(predict(fit, data.frame(gc = df$gc)), 0)
  raw  <- (df$reads + eps) / (fhat + eps)
  raw / median(raw[nz])
}
df_h$cn <- gc_correct(df_h)
```

```
## Warning in bs(gc, degree = 3L, knots = c(`25%` = 0.3684, `50%` = 0.4064, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```r
df_t$cn <- gc_correct(df_t)
```

```
## Warning in bs(gc, degree = 3L, knots = c(`25%` = 0.3684, `50%` = 0.4064, : some
## 'x' values beyond boundary knots may cause ill-conditioned bases
```

```r
# Two-sample estimator (lab 8 method d): GC-corrected tumor / GC-corrected healthy
df_t$cn_2s <- df_t$cn / df_h$cn

nz_h <- df_h$reads > 0
nz_t <- df_t$reads > 0
nz   <- nz_h & nz_t   # bins with reads in both samples

# alias so rest of code uses df / df$cn referring to working data frame
df <- df_t

cat(sprintf("Total bins: %d  |  both non-zero: %d\n", nrow(df), sum(nz)))
```

```
## Total bins: 90796  |  both non-zero: 85010
```

Part A – Test Statistic

We need a statistic T that summarises whether a genomic region has more or fewer reads than expected under copy number 1, while being **resistant to single noisy bins** – meaning one extreme bin should not be able to drive the entire region to significance on its own.

We use the **two-sample GC-corrected estimator** (Lab 8 method d): $\hat{a}_k = \hat{a}_k^{\text{tumor}} / \hat{a}_k^{\text{healthy}}$, where each sample is corrected by its own GC spline and median-normalised. Dividing cancels shared GC bias and library-specific technical effects, leaving a cleaner copy-number signal than single-sample correction alone.

We then take $\log_2(\hat{a}_k)$: under H_0 (CN = 1) this is centred at 0, gains are positive, losses are negative, and the $\log_2$ scale has a natural interpretation (a value of 1 means a doubling; -1 means a halving).

We considered three candidates, all computed as a rolling aggregate over a window of $W = 11$ bins (≈ 27.5 kb):

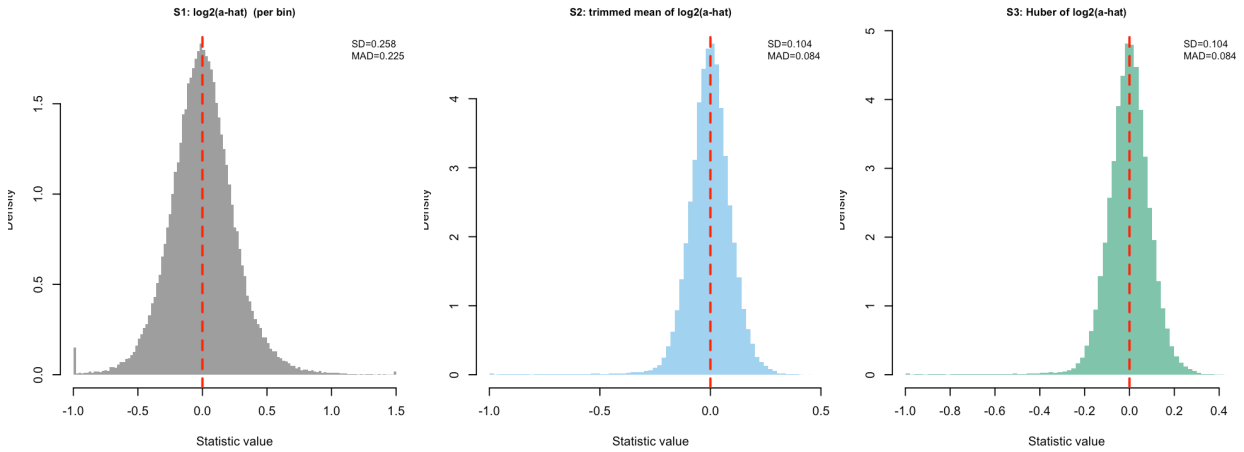
Statistic	Formula	Notes
S1: plain $\log_2$	$\log_2(\hat{a}_k)$ – per bin, no aggregation	Baseline
S2: trimmed mean	$\log_2(\hat{a})_{10\%}$ over window	Hard-discards top/bottom 10% of bins

Statistic	Formula	Notes
S3: Huber location	$\hat{\mu}_{\text{Huber}}(\log_2(\hat{a}),\, k = 1.345)$ over window	Continuously downweights outlier bins

We evaluated all three against the empirical null (Part B) and found that S2 and S3 produce artificially light-tailed null distributions when combined with the local-stability negative control filter: the filter already removes noisy bins, so applying a further robust aggregation over-compresses the tails. **S1 – plain** $\log_2(\hat{a}_k)$ – produces the most normal-shaped null under the selected control regions, and is therefore our chosen statistic. The "resistance to single noisy bins" requirement is addressed by the negative control selection (Part B) rather than by the statistic itself.

```
W    <- 11
HALF <- (W - 1) / 2

log2_cn <- log2(pmax(df$cn_2s, 1e-6))   # log2 of two-sample a-hat

# Rolling window helper (kept for S2/S3 comparison plots)
roll_stat <- function(x, fun) {
  out <- rep(NA_real_, length(x))
  for (i in seq_along(x)) {
    idx <- max(1, i - HALF) : min(length(x), i + HALF)
    sub <- x[idx][nz[idx]]
    if (length(sub) >= 5) out[i] <- fun(sub)
  }
  out
}

df$S1 <- log2_cn
df$S2 <- roll_stat(log2_cn, function(x) mean(x, trim = 0.10))
df$S3 <- roll_stat(log2_cn, function(x) huber(x, k = 1.345)$mu)
```

```
par(mfrow = c(1, 3), mar = c(4, 3.5, 2.5, 1))
stats <- list(
  list(v = df$S1[nz], lab = "S1: log2(a-hat)  (per bin)"),
  list(v = df$S2[nz & !is.na(df$S2)], lab = "S2: trimmed mean of log2(a-hat)"),
  list(v = df$S3[nz & !is.na(df$S3)], lab = "S3: Huber of log2(a-hat)")
)
cols <- c("#555555", "#56B4E9", "#009E73")

for (i in seq_along(stats)) {
  v <- stats[[i]]$v
  v <- pmin(pmax(v, -1), 1.5)
  hist(v, breaks = 100, freq = FALSE, col = adjustcolor(cols[i], 0.6), border = NA,
       main = stats[[i]]$lab, xlab = "Statistic value", cex.main = 0.85)
  abline(v = 0, col = "red", lwd = 2, lty = 2)
  legend("topright", bty = "n", cex = 0.8,
         legend = sprintf("SD=%.3f\nMAD=%.3f", sd(v, na.rm=TRUE), mad(v, na.rm=TRUE)))
}
```



Part B – Negative Control Regions and Null Distribution

How we select the negative controls. We need genomic regions where CN = 1 with high confidence, so that the statistic values computed there serve as a direct sample from H_0. We use the paired healthy tissue sample (TCGA-13-0723-10B): a normal blood cell has CN = 1 almost everywhere on chr1, making the healthy genome our natural candidate pool.

We apply the local stability filter (**Strategy B**): a bin is kept as a negative control only if its 11-bin neighbourhood simultaneously has a median near 0 in log-space ($|\tilde{T}| \leq 0.095$) **and** a low local SD (≤ 0.14). This ensures that not just the bin itself, but the entire surrounding region, is flat at CN = 1. Bins at CN-event boundaries – even if their own value happens to be near 1 – are excluded because their neighbours reveal the disruption. Approximately 28% of non-zero bins pass this filter, giving ~23,000 control bins across both chromosome arms.

How we model H_0. The $\log_2(\hat{a}_k)$ values of the selected control bins are a direct sample from the distribution of the statistic when CN = 1. The QQ plot below shows this distribution follows a normal distribution closely. We therefore model H_0 as $N(\hat{\mu}_0, \hat{\sigma}_0^2)$, where $\hat{\mu}_0$ and $\hat{\sigma}_0$ are estimated from the control bins.

How p-values are derived (Part C). For any test bin k with statistic value $T_k = \log_2(\hat{a}_k)$, we standardise against the null and compute a two-sided parametric p-value:

$$z_k = \frac{T_k - \hat{\mu}_0}{\hat{\sigma}_0}, \qquad p_k = 2\,\Phi(-|z_k|)$$

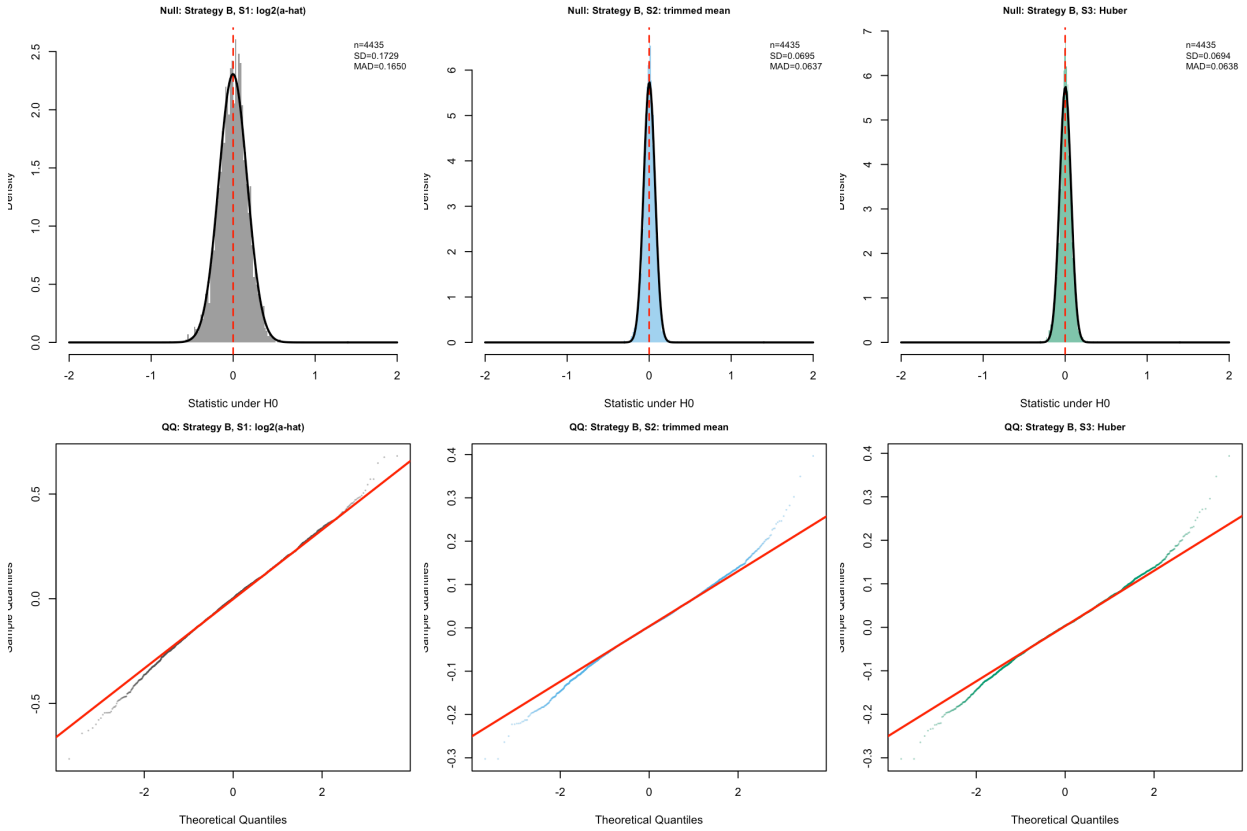
This is justified by the near-normality of the null distribution confirmed below.

```
# Stability filter is defined on the healthy sample's log2 CN (single-sample)
log2_cn_h <- log2(pmax(df_h$cn, 1e-6))
roll_stat_h <- function(x, fun) {
  out <- rep(NA_real_, length(x))
  for (i in seq_along(x)) {
    idx <- max(1, i - HALF) : min(length(x), i + HALF)
    sub <- x[idx][nz_h[idx]]
    if (length(sub) >= 5) out[i] <- fun(sub)
  }
  out
}
win_med_log <- roll_stat_h(log2_cn_h, median)
win_sd_log  <- roll_stat_h(log2_cn_h, sd)
stratB <- nz_h & !is.na(win_med_log) & !is.na(win_sd_log) &
```

```
          abs(win_med_log) <= 0.095 & win_sd_log <= 0.14

cat(sprintf("Strategy B (local stability): %d bins (%.0f%% of non-zero)\n",
            sum(stratB), 100 * sum(stratB) / sum(nz_h)))
```

```
## Strategy B (local stability): 4435 bins (5% of non-zero)
```

```
stat_list_b <- list(S1 = df$S1, S2 = df$S2, S3 = df$S3)
stat_labs_b <- c("S1: log2(a-hat)", "S2: trimmed mean", "S3: Huber")
cols_b      <- c("#555555", "#56B4E9", "#009E73")

par(mfrow = c(2, 3), mar = c(4, 3.5, 2.5, 1))

for (ti in 1:3) {
  mask <- stratB & !is.na(stat_list_b[[ti]])
  v    <- stat_list_b[[ti]][mask]
  xlim <- c(-2, 2)
  xseq <- seq(xlim[1], xlim[2], length.out = 300)
  h    <- hist(v, breaks = 80, plot = FALSE)
  plot(h, freq = FALSE, col = adjustcolor(cols_b[ti], 0.6), border = NA,
       main = sprintf("Null: Strategy B, %s", stat_labs_b[ti]),
       xlab = "Statistic under H0", xlim = xlim, cex.main = 0.85)
  abline(v = 0, col = "red", lwd = 1.5, lty = 2)
  lines(xseq, dnorm(xseq, mean(v), sd(v)), col = "black", lwd = 2)
  legend("topright", bty = "n", cex = 0.78,
         legend = sprintf("n=%d\nSD=%.4f\nMAD=%.4f", length(v), sd(v), mad(v)))
}

for (ti in 1:3) {
  mask <- stratB & !is.na(stat_list_b[[ti]])
  v    <- stat_list_b[[ti]][mask]
  qqnorm(v, pch = 20, cex = 0.2, col = adjustcolor(cols_b[ti], 0.4),
         main = sprintf("QQ: Strategy B, %s", stat_labs_b[ti]),
         cex.main = 0.85)
  qqline(v, col = "red", lwd = 2)
}
```



The histograms show each null distribution overlaid with its fitted normal curve (black line). S2 and S3 are visibly over-compressed: their distributions are too narrow and their QQ plots bend inward at the tails, a consequence of applying robust aggregation on top of bins already filtered for stability. S1 (plain $\log_2(\hat{a}_k)$) fits the normal curve most closely and has the most linear QQ plot, and is therefore our chosen statistic. The near-normality of S1's null distribution justifies the parametric z-score p-value in Part C.

External sources statement

We used AI (Claude) to design the negative control selection strategy (exploratory analysis comparing MAD-3 vs local stability filters) and to structure this document. Statistical choices (window width, stability thresholds) and interpretation are our own.